LEWG, SG14: P0039R0
2015-09-11
Brent Friedman
fourthgeek@gmail.com

Extending `raw_storage_iterator`

I. Motivation

Management of uninitialized memory is an important topic for those implementing containers, allocators, and similar library facilities. This paper seeks to modernize raw storage iterator, bringing important missing features to this utility class.

II. Summary

Move construction

`raw_storage_iterator` lacks support for move construction of elements. Currently users will be faced with the surprising behavior of copy construction in all circumstances.

```
*it = std::move(x); //copy constructs using x
```

Factory function

raw_storage_iterator requires two template parameters which make its usage fairly verbose. We propose a factory function similar to `make_shared` for improving readability and making its use less error prone.

Placement new support

The primary use of `raw_storage_iterator` is to serve as a helper for constructing objects in place. Despite this, it does not support placement new syntax. Support for placement new into a `raw_storage_iterator` makes this iterator useful in new contexts.

III. Discussion

Comments at Lenexa stated that `raw_storage_iterator` is obscure and underused. Fixing these holes should at least open room for this class to be utilized more frequently and to exhibit expected behavior.

No facilities are provided for conditional move, as with move_if_noexcept. The structure of this class would require an understanding of `move_if_noexcept` to be built-in to the type system and so seems to have no good avenue for pursuit. Users of `raw_storage_iterator` should use `move_if_noexcept` at the callsite as they would with any other iterator:

```
*it = move_if_noexcept(v);
```

IV. Proposed Text

Make the following changes in [storage.iterator]:

```
template<class T>
auto make_storage_iterator( T&& iterator)
{
  return raw_storage_iterator<std::remove_reference<T>::type, decltype(*iterator)>( std::forward<T>(iterator));
}

template<class T, class U>
void* operator new(size_t s, raw_storage_iterator<T,U> it) noexcept
{
  return ::operator new(s, it.base() );
}

template<class T, class U>
void operator delete ( void* m, raw_storage_iterator<T,U> it) noexcept
{
  return ::operator delete(m, it.base() );
}
```

Add to `raw_storage_iterator`:

```
raw_storage_iterator& operator=(T&& element);
```

Effects: Move-constructs a value from element at the location to which the iterator points.
Returns: A reference to the iterator.

Amend operator=(const T& element) as follows:

Effects: _Copy-_constructs a value from element ...